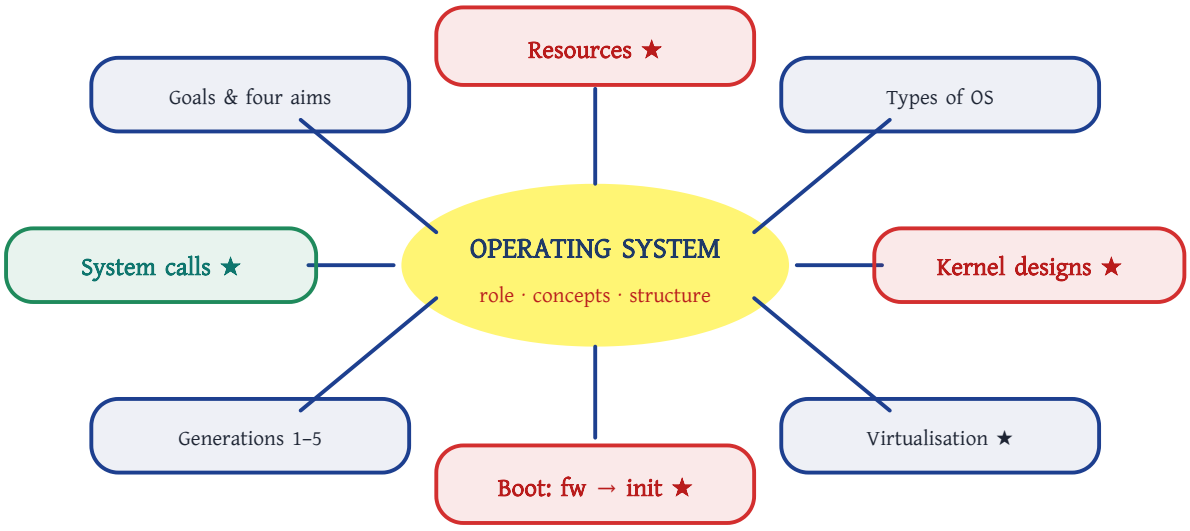
Operating System

Goals · System calls · Kernel structures · Boot · Virtualisation: exam-ready in 12 sheets



↑ the whole topic on one page: red = high-yield, green = the door the OS revolves around

Syllabus coverage

The first two lines of the Operating System syllabus are "**its role in computer systems**" and "**OS concepts; OS structure**". This notebook covers exactly those two. Processes, scheduling, IPC, memory, I/O, deadlock, files and case studies have lessons of their own.

High-yield list

① OS = interface + manager + control
② Goals: easy, full, evolve, portable
③ Kernel privileged; shell only a UI
④ Call ⇒ user → kernel via a **trap**
⑤ Monolithic: all in kernel space
⑥ Microkernel: IPC + scheduling only
⑦ Hybrid: Windows NT, macOS XNU
⑧ Type 1 = bare metal, Type 2 = host
⑨ Boot: fw → loader → kernel → init
⑩ Context switch ≈ 1–10 µs

Legend: **yellow** = key term · **red circle** = must-memorise · **purple box** = memory trick · **red box** = exam trap · **green** = worked example

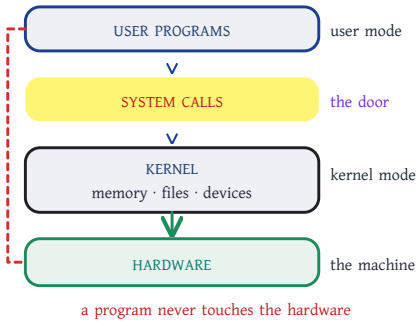
Contents

✓ What an OS is: goals & roles … p. 2
✓ System calls & the mode switch … p. 3
✓ Kernel structure: monolithic → micro … p. 4
✓ Kernel, shell and utilities … p. 5
✓ Types of operating systems … p. 6
✓ Generations of operating systems … p. 7
✓ VM structure & the hypervisor … p. 8
✓ Boot: firmware → init … p. 9
✓ Traps, interrupts, switch cost … p. 10
✓ Night-before sheet … p. 11
✓ Practice MCQs + written … p. 12

1 What Is an Operating System?

Definition: An **operating system** is system software sitting between the user and the hardware, doing three jobs at once: an **interface** (a program asks for a service instead of driving the device), a **resource manager** (CPU, memory, files and I/O shared fairly) and a **control program** (hardware access is privileged, so only the OS may do it). Two textbook names for one object: the **extended machine** and the **resource-manager system**.

> no OS ⇒ only a circuit



one call crosses one boundary

Two classic views of the same program: **Resource manager / control program** — answers *who gets what, and when*: CPU time by scheduling, memory by allocation, files by the file system, devices by controllers and drivers. It optimises **efficiency**. **Extended machine** — answers *what the program sees*: an abstract machine of files, processes and devices, with interrupts, DMA and cache protocols hidden underneath. It optimises **convenience** and **portability**.

Goal	What it demands	Where you meet it
Convenience	a usable interface, not machine code	system calls, CLI shell, GUI, drivers
Efficiency	resources used at full capacity	scheduling, paging, buffering, caching
Ability to evolve	new hardware absorbed without a rewrite	modularity, layers, loadable modules
Portability	the same OS on different hardware	standard language (C), hardware abstraction layer

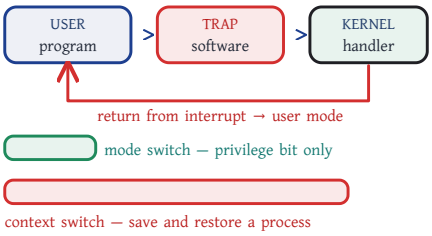
Memory trick: Four Gs: **C**onvenience = *easy*, **E**fficiency = *full*, **E**volution = *changeable*, **P**ortability = *moves house*. "Which goal does paging serve?" → **efficiency**: it raises utilisation, not comfort. "Which does a hardware abstraction layer serve?" → **portability**.

Exam trap: A **driver manages one device; the OS manages all devices and decides who gets them.** And "the interface" is ambiguous: the **shell** is the interface to the user, the **kernel** is the OS proper. "Which part runs privileged?" → **kernel**, never the shell.

Worked example ✍️ goals applied to one machine: One CPU, 8 GB RAM, one disk, four users. **Convenience** — each user gets a shell and named files instead of programming the disk controller. **Efficiency** — the scheduler time-slices so the CPU never idles; the file system caches hot blocks. **Evolution** — a new printer needs one driver, not a new OS. "Which goal did *virtual memory* serve?" → **efficiency**.

2 System Calls, Traps and the Mode Switch

Definition: A **system call** is the interface through which a user program requests a service from the kernel — open a file, map memory, create a process, read a device. The kernel is the only code that runs **privileged**, so the request must change the CPU's privilege level. The mechanism is a **trap** (software interrupt): the CPU saves the context, switches to **kernel mode**, runs the handler, and returns. A **mode switch** that only changes the privilege bit is cheap; a **context switch** that also loads another process's state is not.



"switch" hides two very different costs

Group of system calls	Examples
Process control	fork, execve, wait, exit, kill
File operations	open, close, read, write, lseek
Device management	ioctl, read, write on a device
Information	getpid, uname, sysconf, time
Memory management	brk, mmap, munmap

Point	User mode	Kernel mode
Who runs there	every ordinary process: shell, editor, compiler	scheduler, memory manager, drivers, file system
Memory access	its own address space only	**any** address space, MMIO registers, page tables
Instructions	a restricted subset; privileged ones trap	the complete set (x86 rings 3 → 0)
Entered by	a trap from a call, fault or interrupt	— (it leaves by return-from-interrupt)

Memory trick: Three doors into the kernel: the **trap** (the program asks), the **fault** (the CPU complains — page fault), the **interrupt** (the world asks — timer, keyboard, disk). All three land in kernel mode, the handler chosen by a vector number.

Worked example ✍ what does the overhead cost?: A function call ≈ **20 ns**; a system call with trap and return ≈ **1 µs**. A program issuing 10^6 system calls per second of its own work therefore spends **100 % of one core** inside the kernel.

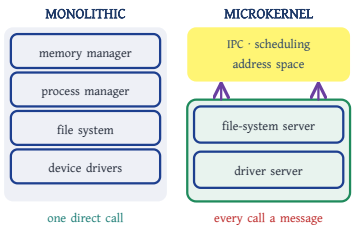
IT link: `man 2 read` = manual page of the *read* syscall; `strace -c ls` counts the traps; `time ls` reports user and system time separately.

Exam trap: A system call is **not** a call between two functions of your own program: the library wrapper (glibc) runs in **user mode** and only the trap enters the kernel.

3 Kernel Structure: Monolithic, Layered, Micro, Modular, Hybrid

Definition: The **kernel** is the privileged core: it runs in kernel mode, stays resident in memory, owns the hardware and implements the system calls. Its internal structure is a design choice — **monolithic**, **layered**, **microkernel**, **modular** — plus the **hybrid** compromise. All answer one question: **how much code is trusted enough to run privileged?**

Structure	Where things live	Strength	Weakness	Example
Monolithic	everything in the kernel	speed, simple call path	one bug stops all	UNIX, Linux, MS-DOS
Layered	all in the kernel, in layers	easy to verify	cost of crossing layers	THE, MULTICS, OS/2
Modular	core + loadable modules	flexible, small images	module bugs not isolated	Linux, Solaris, VxWorks
Microkernel	IPC, scheduling, address space only	reliable, provable	message overhead	Mach, MINIX 3, QNX, seL4
Hybrid	most of it, microkernel-style	speed and modularity	neither simplest nor safest	Windows NT, macOS XNU



same services — only the wall moves

Exam trap: **Hybrid is not "half of each"** — it borrows microkernel modularity but keeps drivers in kernel space for speed. **Modular ≠ microkernel**: a module is kernel code, a microkernel server is user code. A **layer** may use only the layer directly below it.

Memory trick: **"Big kernel, fast calls. Small kernel, chatty calls."** Whatever the drawing style, the only thing that decides performance is whether a request crosses the user/kernel boundary or stays inside one address space.

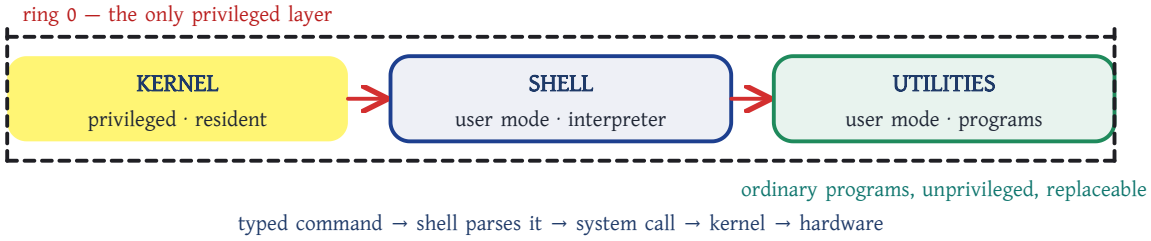
Worked example ✍ price of the small kernel: A microkernel serves a file read as client → **IPC message** → file-system server → **IPC message** → back: at 2 µs per round trip that is **4 µs** of messaging where a monolithic kernel calls directly in under 0.1 µs. At 100 000 reads/s, 40 % of the CPU **goes on messaging**; at 1000 reads/s, 0.4 %.

Where the textbook examples really sit: **Windows NT** and **macOS (XNU)** are hybrid — macOS is the Mach microkernel with BSD services and the IOKit driver framework around it. **Linux** is monolithic but *modularly built*. **seL4** is a microkernel of a few thousand lines that has been **formally verified**.

4 The Three Parts: Kernel, Shell, Utilities

Definition: "Operating system" is used loosely for two different sets of software. Narrowly it is the **kernel** alone: privileged, resident, in charge of the hardware. Broadly it is kernel **+ shell + utilities** — the whole distribution a user touches. The three sit at three different privilege levels, and every question about "is the shell part of the OS?" turns on that difference.

> kernel ≠ shell ≠ utilities



	Kernel	Shell	Utilities
Runs in	**kernel mode**, x86 ring 0	user mode, ring 3	user mode, ring 3
Memory	resident in main memory	an ordinary process, swappable	ordinary processes
Job	processes, memory, files, devices, protection	reads the command line, starts the program	one task each: *cp*, *ls*, *sort*, editors
Replaced by	never, while running	freely — bash, sh, cmd.exe, PowerShell	freely — any program at all
Touches hardware	**yes**	**no**	**no**

Exam trap: **The shell is not the kernel and is not privileged**; it issues system calls for you. Removing it would not crash the machine, corrupting the kernel would. The **compiler** is a utility, not part of the OS.

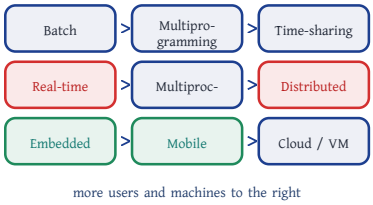
Memory trick: "**The kernel is the one that cannot be replaced.**" Kernel — removing it stops the OS. Shell — you are stranded at a bare login prompt. Utility — you lose one command.

Worked example ✍ trace one keystroke to a file:
cp a.txt b.txt: the shell splits the line and **execve()**s *cp*; *cp* issues **open()**, **read()**, **write()**, **close()**, each trapping into the **kernel**. Privileged code entered: **five times**.

Real-world names: Shell: **bash**, **zsh**, **fish** on UNIX-like systems, **cmd.exe** and **PowerShell** on Windows. Utilities: the **coreutils** family (*ls, cp, mv, grep*) plus editors and compilers. On Linux the shell is a program in `/bin`, replaceable per user.

5 Types of Operating Systems

Definition: An OS is "typed" by **how it shares the processor and whom it serves**, not by its vendor. Three questions fix the type: how many jobs sit in memory, is anybody at a keyboard, must a deadline be met?



one box per promise made

	Hard real-time	Soft real-time
Missed deadline	**failure**	degraded quality
Example	pacemaker, ABS, flight control	streaming video, live TV
Typical build	small dedicated RTOS	general OS + RT extension

Not the same axis: Multiprogramming, time-sharing, multiprocessor and distributed differ in *how many* machines and users; real-time and embedded differ in *what promise* is made. Your phone is all of them at once.

Type	How the CPU is shared	Chief goal	Typical use
Batch	one job at a time, queued off-line	CPU utilisation	mainframe runs
Multiprogramming	many jobs resident; switch on I/O wait	utilisation	base of the next two
Time-sharing	many users online, short slices	response + fairness	UNIX, Linux
Multiprocessor	two or more CPUs share memory	parallelism	servers
Distributed	independent computers, one machine	sharing + reliability	clusters
Real-time	priority, bounded response	predictability	control, avionics
Embedded	one dedicated chip, tiny kernel	determinism, low power	car ECUs
Mobile	app sandboxes, flash storage	user experience	Android, iOS

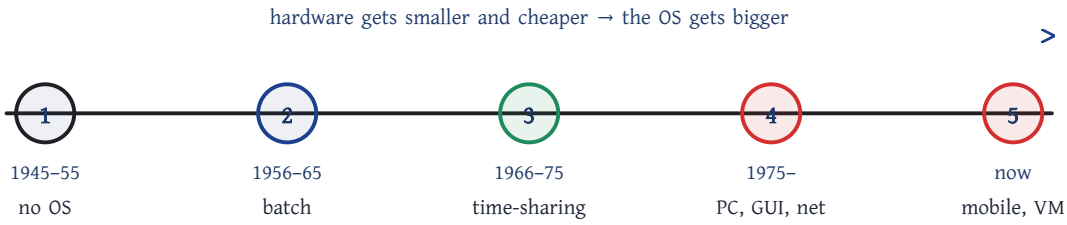
Memory trick: **Time-sharing shares among people; real-time shares among deadlines.** A time-shared UNIX can still miss a 1 ms deadline: fairness and predictability pull opposite ways.

Exam trap: **Real-time does not mean fast** — it means **predictable**. Batch is not "single-user": multiprogramming is what lets several jobs share a CPU.

Worked example ✍️ the time-sharing trade-off:
100 users, quantum q: worst-case wait = 100 × q. **q = 100 ms → 10 s**; **q = 20 ms → 2 s**, but switches now happen every 0.2 ms, so at 2 µs each the CPU loses **1 %** of its time.

6 Generations of Operating Systems

Definition: Operating systems are dated by the **hardware generation they were built for.** Five generations are standard: the first had **no OS at all**, and each later one bought a single new capability.



tube → transistor → IC → microprocessor → everywhere

Gen	Years	Technology	What the OS gained	Landmark
1st	1945–1955	vacuum tubes, paper tape	**nothing**: one job, machine language	IBM 701, EDSAC (1948)
2nd	1956–1965	transistors	**batch**: off-line input, spooling, fixed jobs	IBM 7090; FORTRAN 1957, COBOL 1959
3rd	1966–1975	integrated circuits	**multiprogramming + time-sharing**	UNIX (1969), MULTICS, System/360 (1964)
4th	1975–	VLSI, microprocessors	PC era: GUI, networking, client–server	MS-DOS (1981), Windows, Linux (1991)
5th	present	very large-scale integration, ubiquitous devices	mobile, embedded, virtualised, cloud	Android (2008), hypervisors

Memory trick: **None → batch → sharing → GUI/network → mobile/cloud.** Say the technologies aloud — **tube, transistor, IC, microprocessor, everywhere** — and the generation and its dates fall out.

Exam trap: Generations are **not** types: batch is a 2nd-generation *type*, time-sharing a 3rd-generation *type*. A paper offering both as rival answers is testing this confusion.

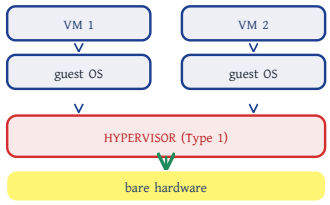
Worked example ✍ reading a clue chain:
"Several jobs in memory, interactive users, integrated circuits, famous system of 1969." → **multiprogramming → time-sharing → 3rd generation → UNIX.** The clue chain beats the dates.

One historical detail worth knowing: UNIX was written in 1969 for a DEC **PDP-7** by Ken Thompson and Dennis Ritchie, rewritten in C in 1970. Generation labels describe the *era*, not one machine.

7 Structural Models: Extensible, VM & the Hypervisor

Definition: The **extensible / VM-based** model keeps the OS tiny: the kernel only traps, schedules and switches, while every user runs a complete little OS on top as a **virtual machine**. That gives fault isolation — a crash kills one VM, not the system — and it is the ancestor of the **hypervisor** (also **VMM**).

Model	Idea	Used by
Simple (MS-DOS-like)	one program, no protection	MS-DOS, early CP/M
Monolithic	every service in one kernel block	UNIX, Linux
Layered	layers, each using only the one below	THE, MULTICS, OS/2
Microkernel	minimal kernel, user-level servers	Mach, MINIX 3, QNX, seL4
Modular	core plus loadable modules	Linux, Solaris, VxWorks
Hybrid	microkernel ideas, monolithic speed	Windows NT, macOS XNU
Extensible / VM	tiny OS; each user runs a whole OS	IBM VM/CMS, ES/VM; Xen, KVM



each guest believes it owns the machine

Point	Type 1 — native / bare-metal	Type 2 — hosted
Runs on	**directly on the hardware**	**on top of a host OS**, as an app
Needs a host OS?	no	yes — Windows, macOS, Linux
Speed	near-native, thin layer	slower: host kernel in the path
Who uses it	data centres, clouds, servers	desktops, labs, training
Examples	Xen, KVM, ESXi, Hyper-V, Proxmox	VirtualBox, Workstation, Parallels

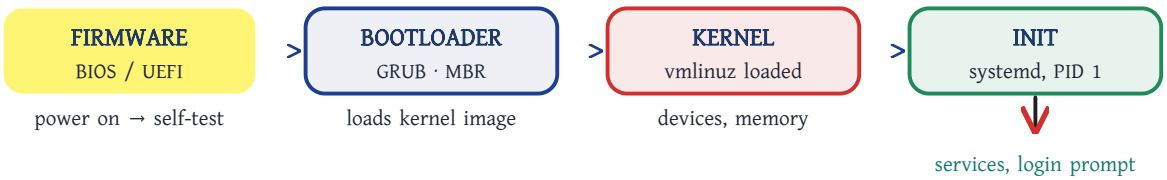
Memory trick: "**Type 1 owns the floor, Type 2 asks the landlord.**" Installed *into* an existing OS → Type 2.

Exam trap: Do not confuse **hypervisor** with **container**: a container shares the host **kernel**, a VM ships a whole guest OS.

Worked example ✍ consolidation arithmetic: 20 old servers at 20 % load, each application in its own VM: 20 × 20 % = 400 % of one core, i.e. **4 cores**. One 32-core host at 15 % leaves **19 servers free**. **KVM** makes the Linux kernel itself a Type 1 hypervisor; **Xen** is paravirtualised.

8 The Boot Process: Firmware → Bootloader → Kernel → init

Definition: A stored-program machine does nothing until an address holds an instruction and the program counter points at it. The **boot sequence** is the ladder that puts the first instruction in the CPU and hands the machine to the OS, one owner at a time.



at every rung exactly one program is in charge

each one loads the next, then gets out of the way

Stage	What it does	Where it lives	Examples
1. Firmware	power-on self-test, initialise the hardware, find a boot device	a chip on the motherboard; settings in CMOS	legacy **BIOS**; modern **UEFI**
2. Bootloader	load the kernel image into RAM and jump to it	first sector of the disk — **MBR** or the **ESP** partition	GRUB, Windows Boot Manager, U-Boot
3. Kernel	initialise subsystems, set up memory, probe devices, start **init**	loaded into RAM, now resident	Linux, Windows NT, BSD
4. init / systemd	start services in order, mount filesystems, run the login manager	the first user-space process, **PID 1**	systemd (modern Linux), SysV init, launchd

Memory trick: "**Firmware sells the machine, the bootloader loads the OS, the kernel owns the machine, init starts the services.**" Four verbs, four owners — only stage 3 is a *kernel*.

Exam trap: **The bootloader is not the OS**, and **BIOS is not the firmware's only name** — UEFI is the successor and uses the ESP, not the MBR. A **kernel panic** is a stage-3 failure.

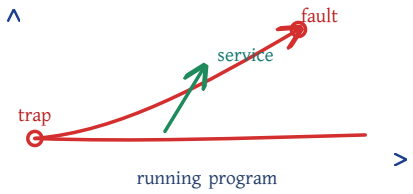
Worked example ✍ why an examiner cares:
"Logo and nothing else." No POST beep → failure **before** the bootloader (RAM, CPU, power). "No bootable device" → the firmware ran and found none. No GRUB menu → the bootloader is damaged.

IT link: Dual boot is the bootloader earning its keep: GRUB shows a menu and loads a different kernel — so it is *not* part of any OS, and one bootloader can boot Windows and Linux from the same disk.

9 Traps, Interrupts and the Cost of a Context Switch

Definition: Control reaches the kernel three ways. A **trap** is synchronous — the program caused it (system call, page fault, divide by zero). An **interrupt** is asynchronous — a device caused it (timer, keypress, disk). A **fault** is the CPU reporting a bad instruction. All save the context, take a **vector** from the vector table and run a handler in kernel mode.

Kind	Caused by	When	Example
Trap	the running program	synchronous	read(), exit(), breakpoint
Fault	the instruction itself	synchronous	page fault, divide by zero
Interrupt	an external device	asynchronous	timer tick, keyboard, disk



time is charged where the trap is taken

Operation	Cost	Where the time goes
Function call	≈ 1–10 ns	push registers, jump, return; caches stay warm
System call (trap)	≈ 1 µs	save context, enter kernel mode, dispatch, return
Context switch	≈ **1–10 µs**	save + restore a PCB; TLB and caches cold after
Major page fault	≈ 100 µs – 1 ms	disk read for the missing page

What a context switch really costs: Saving one process's CPU state and restoring another's is only the **direct** cost, typically **1–10 µs**. The **indirect** cost is larger: the new process meets an empty instruction cache, a cold TLB and cold data caches. More syscalls or a shorter quantum mean more switches.

Memory trick: "**Trap is the program's fault; interrupt is the world's.**"

Exam trap: A **page fault is not a crash** — the OS pages the frame in and restarts the instruction. A **trap is not a context switch**: that means a *different* process ran.

Worked example ✍ throughput lost to switching:
50 users, quantum **20 ms**, switch **2 µs**. Average wait ≈ (50 − 1)/2 × 20 ms ≈ **490 ms**. Overhead per slice = 2/20 000 = **0.01 %** — the design point.

IT link: `time` splits a program's life into **user** and **system** time; system time is time in the kernel. High system time with low CPU use is the signature of **thrashing**.

Night-Before-the-Exam Sheet

Fact sheet:

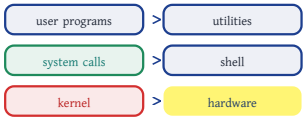
OS = interface + resource manager + control program	Goals: convenience, efficiency, evolve, portability
Kernel = privileged, resident, always in RAM	Shell = user-mode command interpreter
System call ⇒ user → kernel mode via a trap	x86: user = ring 3, kernel = ring 0
Monolithic: all in kernel space (UNIX, Linux)	Microkernel: IPC + scheduling + address space
Hybrid: Windows NT, macOS XNU	Modular: Linux, Solaris
Type 1 hypervisor = bare metal	Type 2 = on a host OS (VirtualBox)
Boot: firmware → bootloader → kernel → init	init / systemd is PID 1
Trap synchronous · interrupt async	Context switch ≈ 1–10 µs; call ≈ 20 ns

Top 12 one-liners for MCQs

1. OS = an interface **and** a resource manager
2. Four goals: convenience, efficiency, evolve, portability
3. Kernel = privileged core; shell = only a UI
4. System call switches user → kernel mode via a trap
5. Monolithic: file systems and drivers in kernel space
6. Microkernel: only IPC, scheduling, address space
7. Hybrid kernel = Windows NT, macOS XNU
8. Type 1 on bare metal; Type 2 on a host OS
9. Boot: firmware → bootloader → kernel → init
10. init (systemd) is the first process, PID 1
11. Trap synchronous, interrupt asynchronous
12. Real-time means predictable, not fast

Top traps: The shell is not the kernel · a driver manages one device, the OS all of them · generations are not types · real-time is not "fast" · a page fault is not a crash · a trap is not a context switch · a container is not a VM.

All tricks together: Four Gs: convenient = easy, efficient = full, evolve = changeable, portable = moves · the kernel cannot be replaced · trap = the program's fault, interrupt = the world's · big kernel = fast calls, small kernel = chatty calls · Type 1 owns the floor, Type 2 asks the landlord.



everything hangs on the middle arrow

Worked example ✍️ one number changes the answer: 1000 requests/s through a microkernel at 2 µs × 2 messages = 4 ms/s = **0.4 %** overhead; at 100 000 requests/s it is 40 %. The same architecture, the wrong answer — structure questions are load questions.

✓ Practice MCQs (exam pattern, 10 marks)

1. Which part of the OS stays resident and manages the CPU, memory and devices?
(a) Shell (b) Kernel
(c) Compiler (d) Bootloader

2. A system call is:
(a) a hardware fault (b) a call inside one program
(c) the interface to get a kernel service (d) a command to restart the machine

3. Which structure keeps only IPC, scheduling and low-level memory in the kernel, with files and drivers as user servers?
(a) Microkernel (b) Monolithic
(c) Layered (d) Simple (MS-DOS-like)

4. The privilege change that lets a user program reach a kernel service is:
(a) context switch (b) dispatch
(c) trap (d) page replacement

5. A pacemaker that must stimulate the heart within 2 ms runs on a:
(a) batch OS (b) hard real-time OS
(c) time-sharing OS (d) distributed OS

6. Which OS generation had vacuum tubes, no OS and machine language?
(a) first (b) second
(c) third (d) fourth

7. A hypervisor installed directly on the hardware with no host OS is:
(a) Type 1 (b) Type 2
(c) Type 3 (d) a container runtime

8. Which component loads the kernel image into memory at start-up?
(a) the shell (b) the bootloader
(c) init (d) the device driver

9. A page fault is:
(a) a crash that ends the process (b) an asynchronous disk interrupt
(c) a synchronous fault the OS services, then the instruction restarts (d) the scheduler's context switch

10. On Linux the first user-space process (PID 1) is:
(a) the bootloader (b) the shell
(c) init / systemd (d) the file system

Answer key → 1-b · 2-c · 3-a · 4-c · 5-b · 6-a · 7-a · 8-b · 9-c · 10-c

Written question: Define an OS, explain it as a resource manager and as a control program, describe with a labelled diagram how a user program reaches a kernel service through a system call, and compare the monolithic and microkernel structures. 10 marks

1. **Definition (2)** — system software between the user and the hardware, managing CPU, memory, files and devices. 2
2. **Resource manager (2)** — *who gets what and when*: CPU by scheduling, memory by allocation and paging, files by the file system, devices by drivers. 2
3. **Control program (1)** — the only program that touches the hardware; programs are isolated, so a bug in one cannot damage the system. 1
4. **Call path (3)** — program → library routine → **trap** → CPU saves context and switches to **kernel mode** → dispatch from the system-call table → service → return-from-trap. Label: user mode, kernel mode, trap, hardware. 3
5. **Comparison (2)** — **monolithic**: all services in one kernel, fast but one bug can stop the machine (UNIX, Linux). **microkernel**: only IPC, scheduling and address space, files and drivers as user servers — reliable, but it pays message overhead (Mach, QNX). 2

Before the exam ✓

- ✓ Can list the four goals
- ✓ Can name kernel, shell, utilities
- ✓ Can draw the trap path
- ✓ Can name the eight types
- ✓ Can place the five generations
- ✓ Can separate Type 1 from Type 2